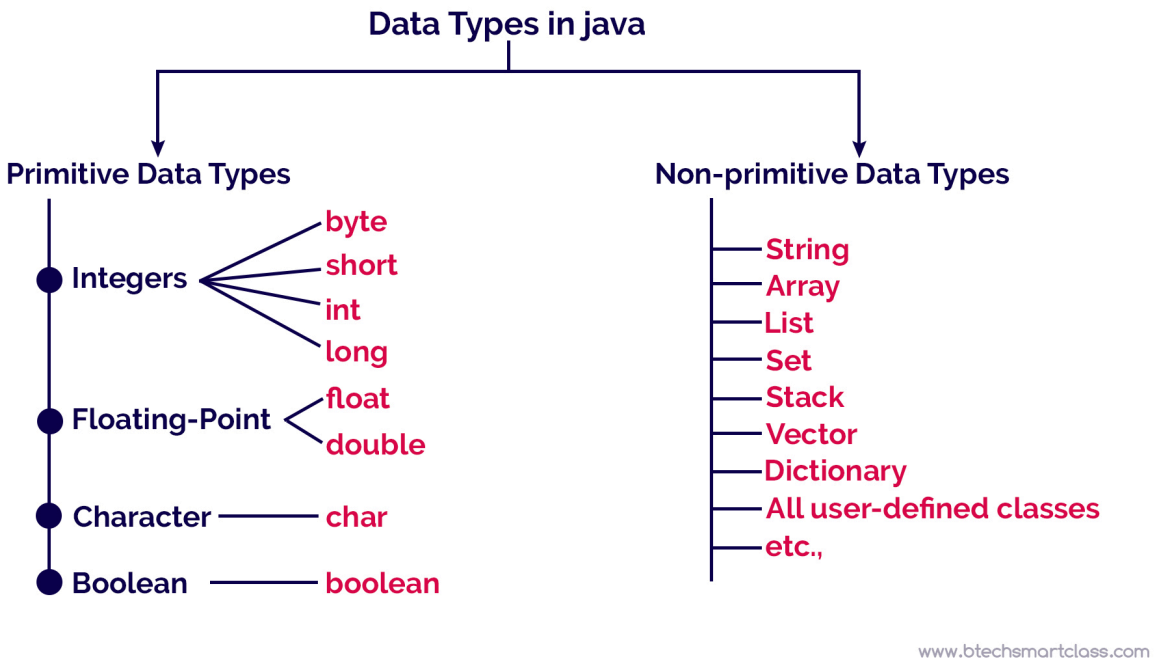
Data Types in Java



Primitive Vs Non-primitive Data Types

Primitive Data Type	Non-primitive Data Type
These are built-in data types	These are created by the users
Does not support additional methods	Support additional methods
Always has a value	It can be null
Starts with lower-case letter	Starts with upper-case letter

Size depends on the data type	Same size for all

TYPE	DESCRIPTION	DEFAULT	SIZE	EXAMPLE LITERALS	RANGE OF VALUES
boolean	true or false	false	1 bit	true, false	true, false
byte	twos complement integer	0	8 bits	(none)	-128 to 127
char	unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\'','\n',' β'	character representation of ASCII values 0 to 255
short	twos complement integer	0	16 bits	(none)	-32,768 to 32,767
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2	-2,147,483,648 to 2,147,483,647
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F	upto 7 decimal digits
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d	upto 16 decimal digits

Reading Assignment:
Primitive vs non Primitive Data types

In Java, type casting is the process of converting a value from one data type to another. This is essential for compatibility when performing operations involving different data types. There are two primary types of casting: widening (implicit) and narrowing (explicit), applicable to both primitive data types and objects within an inheritance hierarchy.

1. Widening Type Casting (Implicit)

Widening casting, also known as automatic type conversion, occurs when converting a smaller data type to a larger one. This process is safe and automatic because the larger type can hold the entire range of values from the smaller type, so no data is lost.

- Process: Automatic (handled by the compiler).

- Safety: Safe; no data loss.
- Example Order: `byte` -> `short` -> `char` -> `int` -> `long` -> `float` -> `double`.

Example:

java

```java
int myInt = 9;
double myDouble = myInt; // Automatic casting from int to double
System.out.println(myInt);    // Output: 9
System.out.println(myDouble); // Output: 9.0
```

2. Narrowing Type Casting (Explicit)

Narrowing casting, also known as manual type conversion, occurs when converting a larger data type to a smaller one. This process is not automatic and requires an explicit cast operator `()` because it can potentially lead to data loss or truncation if the value is too large for the smaller type.

- Process: Manual (requires a cast operator `(targetType)`).
- Safety: Potentially unsafe; data/precision loss can occur.
- Example Order: `double` -> `float` -> `long` -> `int` -> `char` -> `short` -> `byte`.

Example:

java

```java
double myDouble = 9.78;
int myInt = (int) myDouble; // Explicit casting from double to int (truncates decimals)
System.out.println(myDouble); // Output: 9.78
```

```
System.out.println(myInt);    // Output: 9
```

(to be covered in later chapters)

Type Casting with Objects (Reference Types)

Casting also applies to objects within an inheritance hierarchy, allowing you to treat an object as an instance of a different class in the same family.

- Upcasting: Converting a subclass object to a superclass reference. This is always safe and implicit (automatic).
- Downcasting: Converting a superclass reference to a subclass reference. This must be done explicitly and carefully. Using the `instanceof` operator before downcasting is a best practice to avoid a runtime `ClassCastException`.

Casting Between Primitives and Strings

Special methods are used to convert between primitive numeric types and `String` objects, rather than the `()` cast operator.

- Numeric to String: Use `String.valueOf()` or concatenation (`""`).
- String to Numeric: Use parsing methods like `Integer.parseInt()` or `Double.parseDouble()`. If the string is not a valid numeric representation, a `NumberFormatException` will be thrown.

Automatic type promotion, also known as widening primitive conversion or implicit type casting, is a Java feature where a value of a smaller data type is automatically converted to a value of a larger, compatible data type without explicit casting by the programmer. This process is safe as there is no risk of data loss.

When Automatic Type Promotion Occurs

Automatic type promotion happens in several scenarios:

- Assignment to a Larger Type: When a value of a smaller data type is assigned to a variable of a larger data type.
    - *Example:*
    - java

```
int num = 10;

double result = num; // int is automatically promoted to double

System.out.println(result); // Output: 10.0
```

    -
- Arithmetic Expressions: When performing operations involving mixed data types, Java automatically promotes the smaller type(s) to match the largest type in the expression to ensure precision and prevent data loss.
    - *Example:*
    - java

```
int i = 30;

double d = 2.5;

double result = i * d; // 'i' is promoted to double before multiplication

System.out.println("Result is- " + result); // Output: Result is- 75.0
```

    -
- Method Invocation: When an argument passed to a method does not exactly match the parameter type but a wider, compatible type is available.
- Conditional (Ternary) Operator: Operands are promoted to a common type.

Type Promotion Hierarchy

The hierarchy for automatic type promotion, from smallest to largest, is as follows:

`byte` → `short` → `int` → `long` → `float` → `double`

Note that `char` can also be promoted to `int`, `long`, `float`, or `double`. The `boolean` type is not compatible with numeric types and does not support automatic promotion to or from other types.

Rules for Type Promotion in Expressions

Specific rules apply to arithmetic expressions:

- All `byte`, `short`, and `char` operands are automatically promoted to `int` for evaluation.
- If any operand is `long`, the entire expression result is `long`.
- If any operand is `float`, the entire expression result is `float`.
- If any operand is `double`, the entire expression result is `double`.

A common pitfall is when performing operations on small types:

java

```
byte a = 10;

byte b = 20;
```

```
// byte result = a + b; // Compilation Error! 'a + b' results in an int

int result = a + b; // Correct, result is an int (30)
```

To store the result back into a `byte`, an explicit cast is required: `byte result = (byte)(a + b);`.